14 多媒体

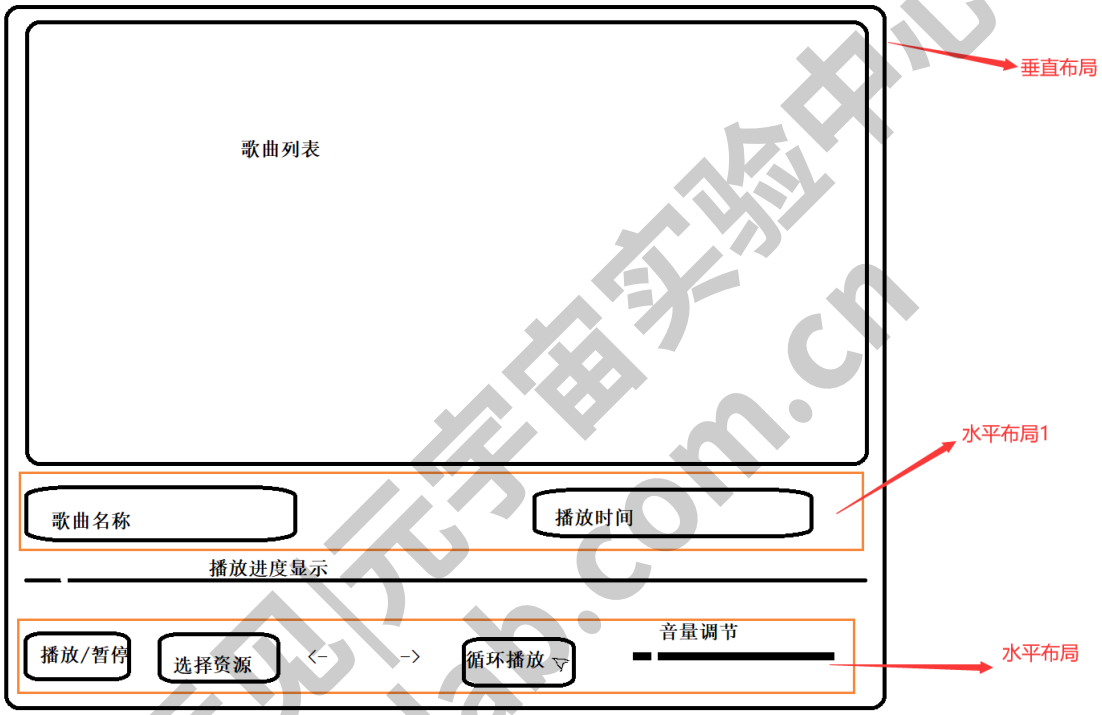
QtMultimedia 模块

1.1 音效播放

mp3音频播放

1.2 音乐播放器

1. 界面设计：使用布局的方式



2. 业务分析
   - 选择歌曲资源，获取到歌曲路径
   - 准备播放器
   - 给播放器设置媒体资源
   - 播放、暂停、继续
   - 播放进度展示： 随着歌曲的播放，播放进度条跟着变化

     ```
     播放进度条的取值范围： [0, 播放总时长]
     如何获取到播放总时长： duration()
     但是要获取到总时长，需要等待一个信号的触发 durationChanged()
     ```

   - 快进、快退：跟着播放滑块的移动改变歌曲播放位置
   - 音量的调节：跟着音量滑块的移动改变播放器的音量
   - 歌曲播放时间的显示：每一秒都要更新 定时器
   - 信号 timeout() 定时器记得启动 start(1000)
   - 歌曲名称的滚动显示

     例如：

华清远见元宇宙实验中心 yyzlab.com.cn

告白气球.mp3
白气球.mp3
气球.mp3
球.mp3

每一秒都显示字符串从某个位置开始的右侧所有
str[start ⇇ step] 字符串切片来实现

要求：

1、先通过简单界面，实现一首歌曲的播放

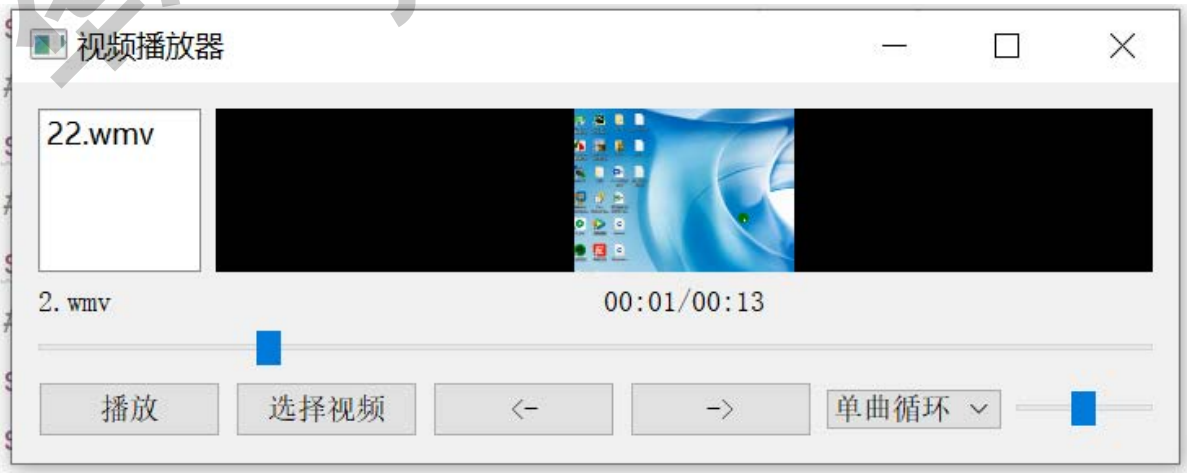
2、通过手写代码的方式，完成界面布局，并实现如图音乐播放器界面及音乐播放功能。

1.3 视频播放器

与音乐播放器的区别就在于多一个画面的显示

视频画面的显示：

```
需要借助 QMediaPlayer中的一个 setVideoOutput()函数
需要借助一个QVideoWidget
```



2. 总结

2.1 QUrl

华清远见|元宇宙实验中心 yyzlab.com.cn

在 PyQt5 中，`QUrl` 是一个用于处理统一资源定位符（URL）的类。它属于 `PyQt5.QtCore` 模块，主要用于解析、操作和验证 URL。`QUrl` 提供了一种跨平台的方式来处理各种形式的 URL（如 HTTP、FTP、文件路径等），并支持对 URL 的各个部分（如协议、主机名、路径等）进行访问和修改。

`QUrl` 的主要作用是解析和操作 URL，通常用于以下场景：

- **解析 URL**：将字符串形式的 URL 解析为可操作的对象。
- **构建 URL**：动态生成 URL。
- **验证 URL**：检查 URL 是否有效。
- **提取 URL 部分**：获取 URL 的协议、主机名、路径、查询参数等。
- **处理文件路径**：将本地文件路径转换为 `file://` 格式的 URL。

2.1.1 `QUrl` 的导入方式

```
from PyQt5.QtCore import QUrl
```

2.1.2 常用方法

2.1.2.1 创建 `QUrl` 对象

- `QUrl(str url)`：通过字符串创建一个 `QUrl` 对象。

  ```
  url = QUrl("https://www.example.com/path?query=123")
  ```

- `QUrl()`：创建一个空的 `QUrl` 对象。

  ```
  url = QUrl()
  ```

2.1.2.2 设置 URL

- `setUrl(str url)`：设置 URL。

  ```
  url.setUrl("https://www.google.com")
  ```

- `setScheme(str scheme)`：设置协议（如 `http`、`https`、`ftp` 等）。

  ```
  url.setScheme("https")
  ```

- `setHost(str host)`：设置主机名。

  ```
  url.setHost("www.example.com")
  ```

- `setPath(str path)`：设置路径。

  ```
  url.setPath("/index.html")
  ```

- `setQuery(str query)`：设置查询参数。

  ```
  url.setQuery("key=value&anotherKey=anotherValue")
  ```

华清远见元宇宙实验中心 yyzlab.com.cn

2.1.2.3 获取 URL 部分

- `url()`：获取完整的 URL 字符串。

```
print(url.url())
```

- `scheme()`：获取协议。

```
print(url.scheme())  # 输出 "https"
```

- `host()`：获取主机名。

```
print(url.host())  # 输出 "www.example.com"
```

- `path()`：获取路径。

```
print(url.path())  # 输出 "/index.html"
```

- `query()`：获取查询参数。

```
print(url.query())  # 输出 "key=value&anotherKey=anotherValue"
```

- `toString()`：将 `QUrl` 转换为字符串。

```
print(url.toString())
```

2.1.2.4 验证 URL

- `isValid()`：检查 URL 是否有效。

```
if url.isValid():
    print("URL 有效")
else:
    print("URL 无效")
```

2.1.2.5 处理文件路径

- `fromLocalFile(str filePath)`：将本地文件路径转换为 `file://` 格式的 URL。

```
file_url = QUrl.fromLocalFile("/path/to/file.txt")
print(file_url.url())  # 输出 "file:///path/to/file.txt"
```

- `toLocalFile()`：将 `file://` 格式的 URL 转换为本地文件路径。

```
local_path = file_url.toLocalFile()
print(local_path)  # 输出 "/path/to/file.txt"
```

华清远见元宇宙实验中心
yyzlab.com.cn

2.1.2.6 处理查询参数

- `queryItems()`：获取查询参数的键值对列表。

```
query_items = url.queryItems()
for key, value in query_items:
    print(f"{key}: {value}")
```

- `addQueryItem(str key, str value)`：添加查询参数。

```
url.addQueryItem("newKey", "newValue")
```

- `removeQueryItem(str key)`：移除指定的查询参数。

```
url.removeQueryItem("newKey")
```

2.1.3 典型应用场景

- **网络请求**：与 `QNetworkRequest` 和 `QNetworkAccessManager` 结合使用，发送 HTTP 请求。
- **文件路径处理**：将本地文件路径转换为 `file://` 格式的 URL，或反之。
- **动态生成 URL**：根据用户输入或其他条件动态生成 URL。
- **解析和提取信息**：从 URL 中提取协议、主机名、路径、查询参数等。
- **验证 URL**：确保用户输入的 URL 是有效的。

2.2 QMediaPlayer

在 PyQt5 中，`QMediaPlayer` 是一个用于播放音频和视频的类。它属于 `PyQt5.QtMultimedia` 模块，主要用于处理多媒体文件（如 MP3、WAV、MP4 等）的播放、暂停、停止等操作。`QMediaPlayer` 提供了对多媒体文件的基本控制功能，并支持与 `QVideoWidget` 或 `QGraphicsVideoItem` 结合使用来显示视频内容。

`QMediaPlayer` 的主要作用是为应用程序提供多媒体播放功能，通常用于以下场景：

- **音频播放**：播放背景音乐或音效。
- **视频播放**：播放本地或网络视频文件。
- **媒体控制**：实现播放、暂停、停止、跳转等功能。
- **状态监控**：实时获取播放状态（如播放中、暂停中、已停止等）。
- **音量控制**：调整播放音量。
- **跨平台支持**：确保代码在不同操作系统上的兼容性。

2.2.1 `QMediaPlayer` 的导入方式

```
from PyQt5.QtMultimedia import QMediaPlayer, QMediaContent
from PyQt5.QtCore import QUrl
```

2.2.2 常用方法

华清远见工业实验中心 www.zlab.com.cn

2.2.2.1 创建 `QMediaPlayer` 对象

- `QMediaPlayer(parent=None)`：创建一个 `QMediaPlayer` 对象。

```
player = QMediaPlayer()
```

2.2.2.2 设置媒体内容

- `setMedia(QMediaContent media)`：设置要播放的媒体内容。
  - 参数：
    - `QMediaContent(QUrl url)`：通过 URL 设置媒体内容。

```
player.setMedia(QMediaContent(QUrl.fromLocalFile("/path/to/audio.mp3")))
```

- `setPlaylist(QMediaPlaylist playlist)`：设置播放列表。

```
from PyQt5.QtMultimedia import QMediaPlaylist

playlist = QMediaPlaylist()
playlist.addMedia(QMediaContent(QUrl.fromLocalFile("/path/to/audio1.mp3")))
playlist.addMedia(QMediaContent(QUrl.fromLocalFile("/path/to/audio2.mp3")))
player.setPlaylist(playlist)
```

2.2.2.3 播放控制

- `play()`：开始播放媒体。

```
player.play()
```

- `pause()`：暂停播放。

```
player.pause()
```

- `stop()`：停止播放。

```
player.stop()
```

2.2.2.4 音量控制

- `setVolume(int volume)`：设置播放音量（范围为 0 到 100）。

```
player.setVolume(50)  # 设置音量为 50%
```

- `volume()`：获取当前音量。

```
print(player.volume())
```

华清远见元宇宙实验中心 yyzlab.com.cn

2.2.2.5 播放位置控制

- `setPosition(int position)`：设置播放位置（单位为毫秒）。

```
player.setPosition(5000)  # 跳转到第 5 秒
```

- `position()`：获取当前播放位置。

```
print(player.position())
```

- `duration()`：获取媒体的总时长（单位为毫秒）。

```
print(player.duration())
```

2.2.2.6 状态监控

- `state()`：获取播放器的状态。
  - 返回值：
    - `QMediaPlayer.StoppedState`：已停止。
    - `QMediaPlayer.PlayingState`：正在播放。
    - `QMediaPlayer.PausedState`：已暂停。

```
if player.state() == QMediaPlayer.PlayingState:
    print("正在播放")
```

- `mediaStatus()`：获取媒体的状态。
  - 返回值：
    - `QMediaPlayer.NoMedia`：没有媒体。
    - `QMediaPlayer.LoadingMedia`：正在加载媒体。
    - `QMediaPlayer.LoadedMedia`：媒体已加载。
    - `QMediaPlayer.BufferingMedia`：正在缓冲媒体。
    - `QMediaPlayer.EndOfMedia`：播放结束。

```
if player.mediaStatus() == QMediaPlayer.EndOfMedia:
    print("播放结束")
```

2.2.2.7 信号与槽

- `stateChanged.connect(slot)`：当播放器状态改变时触发。

```
player.stateChanged.connect(lambda state: print(f"状态改变: {state}"))
```

- `positionChanged.connect(slot)`：当播放位置改变时触发。

```
player.positionChanged.connect(lambda pos: print(f"当前位置: {pos}"))
```

- `durationChanged.connect(slot)`：当媒体时长改变时触发。

```
player.durationChanged.connect(lambda dur: print(f"总时长: {dur}"))
```

华清远见元宇宙实验中心 yyzlab.com.cn

2.2.3 典型应用场景

- **音频播放器**：播放背景音乐或音效。
- **视频播放器**：播放本地或网络视频文件。
- **媒体控制界面**：实现播放、暂停、停止、跳转等功能。
- **播放列表管理**：支持多个媒体文件的连续播放。
- **音量调节**：动态调整播放音量。

2.3 QMediaContent

在 PyQt5 中，`QMediaContent` 是一个用于表示多媒体内容的类。它属于 `PyQt5.QtMultimedia` 模块，主要用于与 `QMediaPlayer` 结合使用，定义要播放的媒体文件或流。`QMediaContent` 提供了一种抽象的方式来处理多媒体资源（如本地文件、网络 URL 或播放列表），并支持跨平台的多媒体播放。

`QMediaContent` 的主要作用是封装多媒体资源，通常用于以下场景：

- **表示本地文件**：将本地文件路径封装为多媒体内容。
- **表示网络资源**：通过 URL 指定网络上的音频或视频资源。
- **支持播放列表**：可以包含多个媒体文件或流。
- **跨平台兼容性**：确保代码在不同操作系统上的兼容性。

2.3.1 `QMediaContent` 的导入方式

```
from PyQt5.QtMultimedia import QMediaContent, QMediaPlaylist
from PyQt5.QtCore import QUrl
```

2.3.2 常用方法

2.3.2.1 创建 `QMediaContent` 对象

- `QMediaContent(QUrl url)`：通过 URL 创建一个多媒体内容对象。

  ```
  media_content = QMediaContent(QUrl.fromLocalFile("/path/to/audio.mp3"))
  ```

- `QMediaContent()`：创建一个空的多媒体内容对象。

```
empty_media = QMediaContent()
```

2.3.2.2 获取资源信息

- `canonicalUrl()`：获取多媒体内容的规范 URL。

  ```
  print(media_content.canonicalUrl().toString())
  ```

- `isNull()`：检查多媒体内容是否为空。

  ```
  if media_content.isNull():
      print("多媒体内容为空")
  ```

华清远见元宇宙实验中心 yyzlab.com.cn

2.3.2.3 结合 `QMediaPlayer` 使用

- `setMedia(QMediaContent media)`：将 `QMediaContent` 设置为 `QMediaPlayer` 的播放内容。

```
player = QMediaPlayer()
player.setMedia(QMediaContent(QUrl.fromLocalFile("/path/to/video.mp4")))
```

2.3.2.4 支持播放列表

- `QMediaPlaylist`：可以将多个 `QMediaContent` 添加到播放列表中。

```
playlist = QMediaPlaylist()
playlist.addMedia(QMediaContent(QUrl.fromLocalFile("/path/to/audio1.mp3")))
playlist.addMedia(QMediaContent(QUrl.fromLocalFile("/path/to/audio2.mp3")))

player = QMediaPlayer()
player.setPlaylist(playlist)
```

2.3.3 典型应用场景

- **本地文件播放**：将本地音频或视频文件封装为 `QMediaContent` 并播放。
- **网络资源播放**：通过 URL 播放网络上的多媒体资源。
- **播放列表管理**：将多个媒体文件封装为播放列表，并实现连续播放。
- **动态切换媒体**：在运行时动态更改播放内容。

2.4 Qt

在 PyQt5 中，`Qt` 是一个核心模块，提供了大量与 GUI 开发相关的类、常量和功能。它是 PyQt5 的基础模块，属于 `PyQt5.QtCore` 和其他子模块的核心部分。`Qt` 提供了许多全局性的定义和工具，例如信号与槽机制、枚举类型、对齐方式、窗口标志等。

`Qt` 的主要作用是为 PyQt5 提供全局性的定义和工具，通常用于以下场景：

- **信号与槽机制**：支持事件驱动的编程模型。
- **枚举类型**：定义了常用的枚举值（如对齐方式、窗口状态、键盘修饰符等）。
- **全局常量**：提供了一些常用的常量（如颜色、对齐方式、按键代码等）。
- **窗口标志**：用于设置窗口的行为和外观（如是否可调整大小、是否全屏等）。
- **跨平台支持**：确保代码在不同操作系统上的兼容性。

2.4.1 `Qt` 的导入方式

```
from PyQt5.QtCore import Qt
```

2.4.2 常用内容

2.4.2.1 信号与槽机制

- **信号与槽** 是 PyQt5 的核心机制，用于实现组件之间的通信。

华南师范大学宇宙实验中心 spacelab.com.cn

```python
from PyQt5.QtWidgets import QPushButton
from PyQt5.QtCore import pyqtSignal, QObject

class Communicate(QObject):
    signal = pyqtSignal()

com = Communicate()
button = QPushButton("点击我")
button.clicked.connect(com.signal.emit)
```

2.4.2.2 对齐方式

- `Qt.AlignmentFlag`：定义了对齐方式的枚举值。
  - 常用值：
    - `Qt.AlignLeft`：左对齐。
    - `Qt.AlignRight`：右对齐。
    - `Qt.AlignCenter`：居中对齐。
    - `Qt.AlignTop`：顶部对齐。
    - `Qt.AlignBottom`：底部对齐。

```python
from PyQt5.QtCore import Qt
from PyQt5.QtWidgets import QLabel

label = QLabel("示例文本")
label.setAlignment(Qt.AlignCenter)  # 居中对齐
```

2.4.2.3 窗口标志

- `Qt.WindowType`：定义了窗口类型的枚举值。
  - 常用值：
    - `Qt.Window`：普通窗口。
    - `Qt.Dialog`：对话框窗口。
    - `Qt.FramelessWindowHint`：无边框窗口。
    - `Qt.WindowStaysOnTopHint`：窗口始终置顶。

```python
from PyQt5.QtCore import Qt
from PyQt5.QtWidgets import QMainWindow

window = QMainWindow()
window.setWindowFlags(Qt.FramelessWindowHint)  # 无边框窗口
```

2.4.2.4 键盘修饰符

- `Qt.KeyboardModifier`：定义了键盘修饰符的枚举值。
  - 常用值：
    - `Qt.ShiftModifier`：Shift 键。
    - `Qt.ControlModifier`：Ctrl 键。
    - `Qt.AltModifier`：Alt 键。

华清远见元宇宙实验中心 yyzlab.com.cn

```
from PyQt5.QtCore import Qt
from PyQt5.QtWidgets import QLabel

def keyPressEvent(event):
    if event.modifiers() == Qt.ControlModifier:
        print("按下了 Ctrl 键")

label = QLabel("按下 Ctrl 键试试")
label.keyPressEvent = keyPressEvent
```

2.4.2.5 鼠标按钮

- `Qt.MouseButton`：定义了鼠标按钮的枚举值。
  - 常用值：
    - `Qt.LeftButton`：左键。
    - `Qt.RightButton`：右键。
    - `Qt.MiddleButton`：中键。

```
from PyQt5.QtCore import Qt
from PyQt5.QtWidgets import QLabel

def mousePressEvent(event):
    if event.button() == Qt.LeftButton:
        print("按下了鼠标左键")

label = QLabel("点击鼠标试试")
label.mousePressEvent = mousePressEvent
```

2.4.2.6 全局常量

- `Qt.GlobalColor`：定义了常用的颜色常量。
  - 常用值：
    - `Qt.red`：红色。
    - `Qt.green`：绿色。
    - `Qt.blue`：蓝色。
    - `Qt.black`：黑色。
    - `Qt.white`：白色。

```
from PyQt5.QtCore import Qt
from PyQt5.QtGui import QPalette
from PyQt5.QtWidgets import QLabel

label = QLabel("示例文本")
palette = label.palette()
palette.setColor(QPalette.WindowText, Qt.red)  # 设置文本颜色为红色
label.setPalette(palette)
```

华清远见元宇宙实验中心 yyzlab.com.cn

2.4.2.7 方向

- `Qt.Orientation`：定义了方向的枚举值。
  - 常用值：
    - `Qt.Horizontal`：水平方向。
    - `Qt.Vertical`：垂直方向。

```
from PyQt5.QtCore import Qt
from PyQt5.QtWidgets import QSlider

slider = QSlider(Qt.Horizontal)  # 创建一个水平滑块
```

2.4.3 典型应用场景

- **信号与槽机制**：实现组件之间的通信。
- **对齐方式**：控制控件或文本的对齐方式。
- **窗口标志**：设置窗口的行为和外观。
- **键盘修饰符**：检测用户按下的键盘修饰键。
- **鼠标按钮**：检测用户点击的鼠标按钮。
- **全局常量**：使用预定义的颜色或其他常量。
- **方向**：指定控件的方向（如水平或垂直）。

2.5 QListWidget

在 PyQt5 中，`QListWidget` 是一个用于显示和管理列表项的控件。它属于 `PyQt5.QtWidgets` 模块，主要用于创建可以滚动的列表视图，并允许用户与列表项进行交互（如选择、双击、拖放等）。`QListWidget` 提供了一个简单的方式来展示一组项目，并支持动态添加、删除和修改列表项。

`QListWidget` 的主要作用是为应用程序提供一个可交互的列表控件，通常用于以下场景：

- **显示列表内容**：展示一组文本或带图标的项目。
- **多选功能**：通过复选框或多种选择模式实现多选功能。
- **动态更新**：可以在运行时动态添加、删除或修改列表项。
- **事件处理**：响应用户的点击、双击等操作。
- **数据绑定**：为每个列表项附加自定义数据。

2.5.1 `QListWidget` 的导入方式

```
from PyQt5.QtWidgets import QListWidget, QListWidgetItem
```

2.5.2 常用方法

2.5.2.1 添加列表项

- `addItem(str label)`：添加一个简单的文本项。

```
list_widget.addItem("示例项")
```

- `addItem(QListWidgetItem item)`：添加一个 `QListWidgetItem` 对象。

华清远见元宇宙实验中心 yy.lab.com.cn

```
item = QListWidgetItem("带图标的项")
item.setIcon(QIcon("icon.png"))
list_widget.addItem(item)
```

- `addItems(list labels)`：一次性添加多个文本项。

```
list_widget.addItems(["项 1", "项 2", "项 3"])
```

2.5.2.2 插入列表项

- `insertItem(int row, str label)`：在指定位置插入一个文本项。

```
list_widget.insertItem(0, "新项")
```

- `insertItem(int row, QListWidgetItem item)`：在指定位置插入一个 `QListWidgetItem` 对象。

```
item = QListWidgetItem("插入的项")
list_widget.insertItem(0, item)
```

2.5.2.3 删除列表项

- `takeItem(int row)`：移除指定位置的列表项。

```
removed_item = list_widget.takeItem(0)
```

- `clear()`：清空所有列表项。

```
list_widget.clear()
```

2.5.2.4 获取列表项

- `item(int row)`：获取指定位置的列表项。

```
item = list_widget.item(0)
print(item.text())
```

- `count()`：获取列表项的总数。

```
print(list_widget.count())
```

2.5.2.5 设置选择模式

- `setSelectionMode(QAbstractItemView.SelectionMode mode)`：设置选择模式。
  - 常用模式：
    - `QAbstractItemView.SingleSelection`：单选模式。
    - `QAbstractItemView.MultiSelection`：多选模式。
    - `QAbstractItemView.ExtendedSelection`：扩展选择模式（支持按住 Ctrl 或 Shift 键选择）。
    - `QAbstractItemView.NoSelection`：禁止选择。

华清远见元宇宙实验中心 yyzlab.com.cn

```
from PyQt5.QtWidgets import QAbstractItemView

list_widget.setSelectionMode(QAbstractItemView.ExtendedSelection)
```

2.5.2.6 获取选中项

- `selectedItems()`：获取所有选中的列表项。

```
selected_items = list_widget.selectedItems()
for item in selected_items:
    print(item.text())
```

- `currentItem()`：获取当前选中的列表项。

```
current_item = list_widget.currentItem()
print(current_item.text())
```

- `currentRow()`：获取当前选中项的行号。

```
row = list_widget.currentRow()
print(row)
```

2.5.2.7 排序列表项

- `sortItems(Qt.SortOrder order=Qt.AscendingOrder)`：对列表项进行排序。
  - 参数：
    - `Qt.AscendingOrder`：升序排序。
    - `Qt.DescendingOrder`：降序排序。

```
list_widget.sortItems(Qt.DescendingOrder)
```

2.5.2.8 信号与槽

- `itemClicked.connect(slot)`：当用户点击列表项时触发。

```
list_widget.itemClicked.connect(lambda item: print(f"点击了: {item.text()}"))
```

- `itemDoubleClicked.connect(slot)`：当用户双击列表项时触发。

```
list_widget.itemDoubleClicked.connect(lambda item: print(f"双击了: {item.text()}"))
```

- `currentItemChanged.connect(slot)`：当当前选中的列表项改变时触发。

```
list_widget.currentItemChanged.connect(lambda current, previous: print(f"当前项: {current.text()}"))
```

华清远见元宇宙实验中心 yyzlab.com.cn

2.5.3 典型应用场景

- **显示列表内容**：在界面中展示一组文本或带图标的项目。
- **多选功能**：通过复选框或多种选择模式实现多选功能。
- **动态更新**：在运行时动态添加、删除或修改列表项。
- **事件处理**：响应用户的点击、双击等操作。
- **数据绑定**：为每个列表项附加自定义数据，方便后续处理。

2.6 QHBoxLayout

在 PyQt5 中，`QHBoxLayout` 是一个用于水平排列控件的布局管理器。它属于 `PyQt5.QtWidgets` 模块，主要用于将控件按照从左到右的顺序排列在一个容器（如窗口或小组件）中。`QHBoxLayout` 提供了一种简单而灵活的方式来管理控件的位置和大小，避免了手动设置控件的几何位置。

`QHBoxLayout` 的主要作用是为应用程序提供水平布局功能，通常用于以下场景：

- **水平排列控件**：将按钮、标签、输入框等控件从左到右排列。
- **自动调整控件大小**：根据窗口大小动态调整控件的宽度和高度。
- **嵌套布局**：与其他布局（如 `QVBoxLayout` 或 `QGridLayout`）结合使用，创建复杂的界面。
- **简化界面设计**：避免手动计算控件的位置和大小。

2.6.1 `QHBoxLayout` 的导入方式

```
from PyQt5.QtWidgets import QHBoxLayout
```

2.6.2 常用方法

2.6.2.1 创建布局

- `QHBoxLayout(parent=None)`：创建一个新的水平布局。

  ```
  layout = QHBoxLayout()
  ```

2.6.2.2 添加控件

- `addWidget(QWidget widget, stretch=0, alignment=0)`：向布局中添加一个控件。
  - 参数：
    - `widget`：要添加的控件。
    - `stretch`：控件的拉伸因子（默认为 0，表示不拉伸）。
    - `alignment`：控件的对齐方式（如 `Qt.AlignLeft`、`Qt.AlignCenter` 等）。

  ```
  from PyQt5.QtWidgets import QPushButton

  button1 = QPushButton("按钮 1")
  button2 = QPushButton("按钮 2")
  layout.addWidget(button1)
  layout.addWidget(button2)
  ```

华清远见元宇宙实验中心 yyzlab.com.cn

2.6.2.3 添加间距

- `addSpacing(int size)`：在布局中添加固定大小的间距。

```
layout.addSpacing(20)  # 添加 20 像素的间距
```

- `addStretch(int stretch=0)`：在布局中添加可伸缩的空白区域。

```
layout.addStretch()  # 添加一个可伸缩的空白区域
```

2.6.2.4 设置控件的拉伸因子

- `setStretch(int index, int stretch)`：设置指定索引处控件的拉伸因子。

```
layout.setStretch(0, 1)  # 第一个控件的拉伸因子为 1
layout.setStretch(1, 2)  # 第二个控件的拉伸因子为 2
```

2.6.2.5 嵌套布局

- `addLayout(QLayout layout, stretch=0)`：向布局中添加另一个布局。

```
from PyQt5.QtWidgets import QVBoxLayout

inner_layout = QVBoxLayout()
layout.addLayout(inner_layout)
```

2.6.2.6 移除控件

- `removeWidget(QWidget widget)`：从布局中移除指定的控件。

```
layout.removeWidget(button1)
```

2.6.2.7 清空布局

- `deleteLater()`：销毁布局及其所有子控件。

```
layout.deleteLater()
```

2.6.3 典型应用场景

- **工具栏**：将按钮或其他控件水平排列在一个工具栏中。
- **表单布局**：将标签和输入框水平排列，形成简单的表单。
- **嵌套布局**：与其他布局结合使用，创建复杂的用户界面。
- **动态调整控件大小**：根据窗口大小自动调整控件的宽度和高度。

2.7 QVBoxLayout

在 PyQt5 中，`QVBoxLayout` 是一个用于垂直排列控件的布局管理器。它属于 `PyQt5.QtWidgets` 模块，主要用于将控件按照从上到下的顺序排列在一个容器（如窗口或小组件）中。与 `QHBoxLayout` 类似，`QVBoxLayout` 提供了一种简单而灵活的方式来管理控件的位置和大小，避免了手动设置控件的几何位置。

`QVBoxLayout` 的主要作用是为应用程序提供垂直布局功能，通常用于以下场景：

华清远见元宇宙实验中心 yyzlab.com.cn

- **垂直排列控件**：将按钮、标签、输入框等控件从上到下排列。
- **自动调整控件大小**：根据窗口大小动态调整控件的宽度和高度。
- **嵌套布局**：与其他布局（如 `QHBoxLayout` 或 `QGridLayout`）结合使用，创建复杂的界面。
- **简化界面设计**：避免手动计算控件的位置和大小。

2.7.1 `QVBoxLayout` 的导入方式

```
from PyQt5.QtWidgets import QVBoxLayout
```

2.7.2 常用方法

2.7.2.1 创建布局

- `QVBoxLayout(parent=None)`：创建一个新的垂直布局。

  ```
  layout = QVBoxLayout()
  ```

2.7.2.2 添加控件

- `addWidget(QWidget widget, stretch=0, alignment=0)`：向布局中添加一个控件。
  - 参数：
    - `widget`：要添加的控件。
    - `stretch`：控件的拉伸因子（默认为 0，表示不拉伸）。
    - `alignment`：控件的对齐方式（如 `Qt.AlignLeft`、`Qt.AlignCenter` 等）。

  ```
  from PyQt5.QtWidgets import QPushButton

  button1 = QPushButton("按钮 1")
  button2 = QPushButton("按钮 2")
  layout.addWidget(button1)
  layout.addWidget(button2)
  ```

2.7.2.3 添加间距

- `addSpacing(int size)`：在布局中添加固定大小的间距。

  ```
  layout.addSpacing(20)  # 添加 20 像素的间距
  ```

- `addStretch(int stretch=0)`：在布局中添加可伸缩的空白区域。

  ```
  layout.addStretch()  # 添加一个可伸缩的空白区域
  ```

2.7.2.4 设置控件的拉伸因子

- `setStretch(int index, int stretch)`：设置指定索引处控件的拉伸因子。

  ```
  layout.setStretch(0, 1)  # 第一个控件的拉伸因子为 1
  layout.setStretch(1, 2)  # 第二个控件的拉伸因子为 2
  ```

北清远见元宇宙实验中心 yyzlab.com.cn

2.7.2.5 嵌套布局

- `addLayout(QLayout layout, stretch=0)`：向布局中添加另一个布局。

```
from PyQt5.QtWidgets import QHBoxLayout

inner_layout = QHBoxLayout()
layout.addLayout(inner_layout)
```

2.7.2.6 移除控件

- `removeWidget(QWidget widget)`：从布局中移除指定的控件。

```
layout.removeWidget(button1)
```

2.7.2.7 清空布局

- `deleteLater()`：销毁布局及其所有子控件。

```
layout.deleteLater()
```

2.7.3 典型应用场景

- **表单布局**：将标签和输入框垂直排列，形成简单的表单。
- **菜单栏**：将按钮或其他控件垂直排列在一个菜单栏中。
- **嵌套布局**：与其他布局结合使用，创建复杂的用户界面。
- **动态调整控件大小**：根据窗口大小自动调整控件的宽度和高度。

2.8 QSlider

在 PyQt5 中，`QSlider` 是一个用于创建滑块控件的类。它属于 `PyQt5.QtWidgets` 模块，主要用于让用户通过拖动滑块来选择一个数值范围内的值。`QSlider` 提供了直观的交互方式，适用于音量调节、进度控制、数值选择等场景。

`QSlider` 的主要作用是为用户提供一个可拖动的滑块，通常用于以下场景：

- **数值选择**：让用户从一个范围内选择一个值。
- **音量调节**：作为音量控制器。
- **进度控制**：显示或控制任务的进度。
- **动态更新**：支持实时响应用户的滑动操作。
- **方向支持**：支持水平和垂直方向的滑块。

2.8.1 `QSlider` 的导入方式

```
from PyQt5.QtWidgets import QSlider
```

2.8.2 常用方法

华清远见|元宇宙实验中心 yyzlab.com.cn

2.8.2.1 创建滑块

- `QSlider(Qt.Orientation orientation, parent=None)`：创建一个滑块。
  - 参数：
    - `orientation`：滑块的方向（`Qt.Horizontal` 表示水平，`Qt.Vertical` 表示垂直）。

```
from PyQt5.QtCore import Qt

slider = QSlider(Qt.Horizontal)  # 创建一个水平滑块
```

2.8.2.2 设置范围

- `setMinimum(int min)`：设置滑块的最小值。

```
slider.setMinimum(0)
```

- `setMaximum(int max)`：设置滑块的最大值。

```
slider.setMaximum(100)
```

- `setRange(int min, int max)`：同时设置滑块的最小值和最大值。

```
slider.setRange(0, 100)
```

2.8.2.3 设置当前值

- `setValue(int value)`：设置滑块的当前值。

```
slider.setValue(50)
```

- `value()`：获取滑块的当前值。

```
print(slider.value())
```

2.8.2.4 设置步长

- `setSingleStep(int step)`：设置单步步长（用户点击箭头时的增量）。

```
slider.setSingleStep(1)
```

- `setPageStep(int step)`：设置页面步长（用户按住滑块拖动时的增量）。

```
slider.setPageStep(10)
```

华清远见元宇宙实验中心
yyzlab.com.cn

2.8.2.5 设置方向

- `setOrientation(Qt.Orientation orientation)`：设置滑块的方向。

```
slider.setOrientation(Qt.Vertical)  # 设置为垂直方向
```

2.8.2.6 启用刻度

- `setTickPosition(QSlider.TickPosition position)`：设置刻度的位置。
  - 常用值：
    - `QSlider.NoTicks`：不显示刻度。
    - `QSlider.TicksAbove`：刻度显示在滑块上方（水平滑块）。
    - `QSlider.TicksBelow`：刻度显示在滑块下方（水平滑块）。
    - `QSlider.TicksLeft`：刻度显示在滑块左侧（垂直滑块）。
    - `QSlider.TicksRight`：刻度显示在滑块右侧（垂直滑块）。

```
slider.setTickPosition(QSlider.TicksBelow)
```

- `setTickInterval(int interval)`：设置刻度的间隔。

```
slider.setTickInterval(10)
```

2.8.2.7 信号与槽

- `valueChanged.connect(slot)`：当滑块的值改变时触发。

```
slider.valueChanged.connect(lambda value: print(f"当前值: {value}"))
```

- `sliderMoved.connect(slot)`：当用户拖动滑块时触发。

```
slider.sliderMoved.connect(lambda position: print(f"滑块位置: {position}"))
```

- `sliderPressed.connect(slot)`：当用户按下滑块时触发。

```
slider.sliderPressed.connect(lambda: print("滑块被按下"))
```

- `sliderReleased.connect(slot)`：当用户释放滑块时触发。

```
slider.sliderReleased.connect(lambda: print("滑块被释放"))
```

2.8.3 典型应用场景

- **音量调节**：作为音量控制器，让用户调整音量大小。
- **进度控制**：显示或控制任务的进度。
- **数值选择**：让用户从一个范围内选择一个值。
- **动态更新**：实时响应用户的滑动操作。
- **刻度显示**：显示刻度以增强用户体验。

2.9 QComboBox

华清远见元宇宙实验中心 yyzlab.com.cn

在 PyQt5 中，`QComboBox` 是一个用于创建下拉列表控件的类。它属于 `PyQt5.QtWidgets` 模块，主要用于让用户从一组选项中选择一个值。`QComboBox` 提供了直观的交互方式，适用于选择菜单、设置选项等场景。

`QComboBox` 的主要作用是为用户提供一个下拉列表，通常用于以下场景：

- **单选功能**：让用户从一组预定义的选项中选择一个值。
- **动态更新**：支持在运行时动态添加、删除或修改选项。
- **可编辑模式**：允许用户输入自定义值（可选）。
- **事件处理**：响应用户的选项选择操作。
- **数据绑定**：为每个选项附加自定义数据。

2.9.1 `QComboBox` 的导入方式

```
from PyQt5.QtWidgets import QComboBox
```

2.9.2 常用方法

2.9.2.1 创建下拉列表

- `QComboBox(parent=None)`：创建一个下拉列表控件。

  ```
  combo_box = QComboBox()
  ```

2.9.2.2 添加选项

- `addItem(str text, userData=None)`：向下拉列表中添加一个选项。
  - 参数：
    - `text`：选项的显示文本。
    - `userData`：与选项关联的自定义数据（可选）。

  ```
  combo_box.addItem("选项 1")
  combo_box.addItem("选项 2", "自定义数据")
  ```

- `addItems(list texts)`：一次性添加多个选项。

  ```
  combo_box.addItems(["选项 A", "选项 B", "选项 C"])
  ```

2.9.2.3 插入选项

- `insertItem(int index, str text, userData=None)`：在指定位置插入一个选项。

  ```
  combo_box.insertItem(0, "新选项")
  ```

2.9.2.4 删除选项

- `removeItem(int index)`：移除指定索引处的选项。

  ```
  combo_box.removeItem(0)
  ```

- `clear()`：清空所有选项。

华清远见元宇宙实验中心 yyzlab.com.cn

```
combo_box.clear()
```

2.9.2.5 获取选项信息

- `count()`：获取选项的总数。

```
print(combo_box.count())
```

- `itemText(int index)`：获取指定索引处选项的文本。

```
print(combo_box.itemText(0))
```

- `currentText()`：获取当前选中的选项文本。

```
print(combo_box.currentText())
```

- `currentIndex()`：获取当前选中的选项索引。

```
print(combo_box.currentIndex())
```

- `itemData(int index)`：获取指定索引处选项的自定义数据。

```
print(combo_box.itemData(1))
```

2.9.2.6 设置当前选项

- `setCurrentIndex(int index)`：设置当前选中的选项索引。

```
combo_box.setCurrentIndex(1)
```

- `setCurrentText(str text)`：设置当前选中的选项文本。

```
combo_box.setCurrentText("选项 2")
```

2.9.2.7 启用可编辑模式

- `setEditable(bool editable)`：启用或禁用可编辑模式。

```
combo_box.setEditable(True)  # 允许用户输入自定义值
```

- `isEditable()`：检查是否启用了可编辑模式。

```
if combo_box.isEditable():
    print("下拉列表是可编辑的")
```

华清远见元宇宙实验中心
yyzlab.com.cn

2.9.2.8 信号与槽

- `currentIndexChanged.connect(slot)`：当当前选中的选项索引改变时触发。

  ```
  combo_box.currentIndexChanged.connect(lambda index: print(f"当前索引: {index}"))
  ```

- `currentTextChanged.connect(slot)`：当当前选中的选项文本改变时触发。

  ```
  combo_box.currentTextChanged.connect(lambda text: print(f"当前文本: {text}"))
  ```

- `activated.connect(slot)`：当用户选择某个选项时触发。

  ```
  combo_box.activated.connect(lambda index: print(f"激活索引: {index}"))
  ```

- `editTextChanged.connect(slot)`：当用户在可编辑模式下修改文本时触发。

  ```
  combo_box.editTextChanged.connect(lambda text: print(f"编辑后的文本: {text}"))
  ```

2.9.3 典型应用场景

- **单选功能**：让用户从一组预定义的选项中选择一个值。
- **动态更新**：在运行时动态添加、删除或修改选项。
- **可编辑模式**：允许用户输入自定义值。
- **事件处理**：响应用户的选项选择操作。
- **数据绑定**：为每个选项附加自定义数据。

2.10 QVideoWidget

在 PyQt5 中，`QVideoWidget` 是一个用于显示视频内容的控件。它属于 `PyQt5.QtMultimediaWidgets` 模块，通常与 `QMediaPlayer` 结合使用，用于播放和显示视频文件或流媒体内容。`QVideoWidget` 提供了一个简单的界面来渲染视频，并支持调整视频的显示属性（如缩放模式、宽高比等）。

`QVideoWidget` 的主要作用是为应用程序提供视频显示功能，通常用于以下场景：

- **视频播放**：显示本地或网络视频文件。
- **嵌入式视频**：将视频嵌入到 GUI 界面中。
- **自定义显示**：调整视频的缩放模式、宽高比等属性。
- **跨平台支持**：确保代码在不同操作系统上的兼容性。

2.10.1 `QVideoWidget` 的导入方式

```
from PyQt5.QtMultimediaWidgets import QVideoWidget
```

2.10.2 常用方法

传清远观光宇宙实验中心 yyzlab.com.cn

2.10.2.1 创建视频控件

- `QVideoWidget(parent=None)`：创建一个视频控件。

```
video_widget = QVideoWidget()
```

2.10.2.2 设置缩放模式

- `setAspectRatioMode(Qt.AspectRatioMode mode)`：设置视频的宽高比模式。
  - 常用值：
    - `Qt.KeepAspectRatio`：保持原始宽高比。
    - `Qt.IgnoreAspectRatio`：忽略宽高比，拉伸以填满控件。
    - `Qt.KeepAspectRatioByExpanding`：保持宽高比，但可能会裁剪部分内容。

```
from PyQt5.QtCore import Qt

video_widget.setAspectRatioMode(Qt.KeepAspectRatio)
```

2.10.2.3 设置全屏模式

- `setFullScreen(bool fullScreen)`：切换视频控件的全屏模式。

```
video_widget.setFullScreen(True)  # 切换到全屏
```

- `isFullScreen()`：检查视频控件是否处于全屏模式。

```
if video_widget.isFullScreen():
    print("视频控件处于全屏模式")
```

2.10.2.4 结合 `QMediaPlayer` 使用

- `setVideoOutput(QVideoWidget videoOutput)`：将视频输出绑定到 `QVideoWidget`。

```
from PyQt5.QtMultimedia import QMediaPlayer, QMediaContent
from PyQt5.QtCore import QUrl

player = QMediaPlayer()
player.setVideoOutput(video_widget)
player.setMedia(QMediaContent(QUrl.fromLocalFile("/path/to/video.mp4")))
player.play()
```

2.10.2.5 信号与槽

- `fullScreenChanged.connect(slot)`：当全屏状态改变时触发。

```
video_widget.fullScreenChanged.connect(lambda is_full_screen: print(f"全屏状态: {is_full_screen}"))
```

华清远见元宇宙实验中心 yyzlab.com.cn

2.10.3 典型应用场景

- **视频播放器**：创建一个完整的视频播放器，支持播放、暂停、停止等功能。
- **嵌入式视频**：将视频嵌入到 GUI 界面中，与其他控件一起使用。
- **自定义显示**：调整视频的缩放模式、宽高比等属性，以满足特定需求。
- **全屏播放**：支持视频的全屏播放模式。

华清远见|元宇宙实验中心
yyzlab.com.cn